



Enterprise Spring Deep Dive

Spring Boot • Spring Core • Data • Security • Microservices

Production-Ready Java Architecture



Session Structure

1. Spring Core fundamentals
2. Dependency Injection & IoC
3. Spring Boot internals
4. Spring Data JPA
5. Spring Security
6. REST best practices
7. Enterprise microservices patterns
8. Performance & production tuning



Why Spring?

Spring provides:

- Dependency Injection
- Simplified configuration
- Enterprise integration
- Production-ready ecosystem

Used in:

Most enterprise Java systems.



Spring Core Concepts

- Inversion of Control (IoC)
- Dependency Injection (DI)
- Bean lifecycle
- ApplicationContext

Dependency Injection Example

```
@Service
class UserService {

    private final UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }
}
```

Constructor injection preferred.



Bean Lifecycle

1. Instantiation
2. Dependency injection
3. @PostConstruct
4. Ready for use
5. @PreDestroy



Spring Boot Auto Configuration

Auto-configures:

- DataSource
- DispatcherServlet
- Jackson
- Security

Reduces boilerplate.



Spring Boot Application

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```




REST Controller Example

```
@RestController
@RequestMapping("/users")
class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return new User(id, "John");
    }
}
```



Spring Data JPA

Repository interface:

```
public interface UserRepository  
    extends JpaRepository<User, Long> {  
}
```

Provides CRUD automatically.



Entity Example

```
@Entity
class User {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
}
```



Transaction Management

```
@Transactional  
public void processOrder() {  
    // DB operations  
}
```

Ensures atomicity.



Spring Security Basics

Authentication

Authorization

Filters

THIRAVUGAL



JWT Authentication Flow

Client → Login

Server → Generate JWT

Client → Send JWT in header

Server → Validate token

Security Configuration

```
@EnableWebSecurity
class SecurityConfig {

    protected void configure(
        HttpSecurity http) throws Exception {

        http.authorizeRequests()
            .anyRequest().authenticated();
    }
}
```

Exception Handling

```
@ControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handle(Exception e) {
        return ResponseEntity
            .status(500)
            .body("Error");
    }
}
```




Microservices with Spring Cloud

- Eureka (Service discovery)
- Config Server
- API Gateway
- Feign Client



Feign Client Example

```
@FeignClient(name = "order-service")
interface OrderClient {

    @GetMapping("/orders/{id}")
    Order getOrder(@PathVariable Long id);
}
```



Circuit Breaker

Using Resilience4j:

Prevents cascading failures.

THIRAVUGAL



Spring Profiles

```
@Profile("dev")  
@Bean  
public DataSource devDataSource() {}
```

Environment-based config.



Logging & Monitoring

Use:

- Spring Boot Actuator
- Micrometer
- Prometheus
- ELK stack



Performance Best Practices

- Use connection pools (HikariCP)
- Avoid N+1 query problem
- Use caching (@Cacheable)
- Enable compression
- Proper thread pool sizing



@Cacheable Example

```
@Cacheable("users")
public User getUser(Long id) {
    return repo.findById(id).orElse(null);
}
```

Reduces DB calls.



Common Enterprise Mistakes

- Field injection
- Large controller logic
- No transaction boundaries
- No logging
- Poor exception handling



Enterprise Architecture Layers

Presentation Layer

Business Layer

Persistence Layer

Integration Layer

Clear separation.



Scaling Spring Applications

- Stateless services
- Externalized config
- Horizontal scaling
- Kubernetes deployment



JVM & Spring Optimization

- Heap tuning
- GC tuning
- Reduce reflection overhead
- Lazy initialization where needed



Real World Production Checklist

- Logging
- Metrics
- Health checks
- Exception handling
- Security hardening
- Load testing
- CI/CD pipeline

Interview-Level Spring Questions

- How does Spring IoC work?
- Difference between `@Component` and `@Service`?
- How does Spring Boot auto-config work?
- What is proxy in Spring?
- How transactions work internally?
- Difference between singleton and prototype bean?

Summary

Enterprise Spring mastery requires:

- Understanding IoC deeply
- Clean layered architecture
- Proper transaction management
- Secure API design
- Performance tuning
- Production readiness

You are now equipped for:

- Enterprise backend roles
- Java developer positions
- Spring-based system architecture
- Production system engineering